

8.First Fit Contiguous Memory Allocation

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Save

First Fit Contiguous Memory Allocation

Completed

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is allocated or not.
- If allocated, it displays the process number, process size, block number, block size, and the internal fragmentation (block size minus process size).
- If not allocated, it indicates that the process cannot be allocated.

Sample Input 1

```
5
100 500 200 300 600
4
212 417 112 426
```

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int m, n;
5
6     printf("Enter number of memory blocks:\n");
7     scanf("%d", &m);
8
9     int blockSize[m], originalBlockSize[m];
10
11     printf("Enter size of each block:\n");
12     for(int i = 0; i < m; i++) {
13         scanf("%d", &blockSize[i]);
14         originalBlockSize[i] = blockSize[i];
15     }
16
17     printf("Enter number of processes:\n");
18     scanf("%d", &n);
19
20     int processSize[n];
21
22     printf("Enter size of each process:\n");
23     for(int i = 0; i < n; i++) {
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Sample Test Case

Test Case - 1

Test Case - 2

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Save

First Fit Contiguous Memory Allocation

Completed

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is allocated or not.
- If allocated, it displays the process number, process size, block number, block size, and the internal fragmentation (block size minus process size).
- If not allocated, it indicates that the process cannot be allocated.

Sample Input 1

```
5
100 500 200 300 600
4
212 417 112 426
```

Select Language: C (GCC 9.2.0)

```
22     printf("Enter size of each process:\n");
23     for(int i = 0; i < n; i++) {
24         scanf("%d", &processSize[i]);
25     }
26
27     for(int i = 0; i < n; i++) {
28         int allocated = 0;
29
30         for(int j = 0; j < m; j++) {
31             if(blockSize[j] >= processSize[i]) {
32
33                 int fragment = blockSize[j] - processSize[i];
34
35                 printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
36                     i + 1, processSize[i], j + 1, originalBlockSize[j], fragment);
37
38                 blockSize[j] = fragment;
39
40                 allocated = 1;
41                 break;
42             }
43         }
44     }
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Sample Test Case

Test Case - 1

Test Case - 2

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Save

First Fit Contiguous Memory Allocation

Completed

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.

2. The next line contains m integers representing the sizes of each memory block.

3. The next line contains an integer n representing the number of processes.

4. The next line contains n integers representing the sizes of each process.

Output Format

• For each process, the program prints whether the process is allocated or not.

• If allocated, it displays the process number, process size, block number, block size, and the internal fragmentation (block size minus process size).

• If not allocated, it indicates that the process cannot be allocated.

Sample Input 1

5

100 500 200 300 600

4

212 417 112 426

Select Language : C (GCC 9.2.0)

30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52

```
for(int j = 0; j < n; j++) {  
    if(blockSize[j] >= processSize[i]) {  
  
        int fragment = blockSize[j] - processSize[i];  
  
        printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",  
            i + 1, processSize[i], j + 1, originalBlockSize[j], fragment);  
  
        blockSize[j] = fragment;  
  
        allocated = 1;  
        break;  
    }  
}  
  
if(allocated == 0) {  
    printf("Process %d of size %d is not allocated\n",  
        i + 1, processSize[i]);  
}  
}  
  
return 0;  
}
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2